

Задание

Разработать backend-приложение для логистической компании, которое позволяет управлять грузами, транспортом и доставками.

Система должна предоставлять REST API для создания и отслеживания доставок, назначения транспорта, расчёта стоимости и формирования базовой отчётности.

Требуется использовать Java и Spring Boot.

Особое внимание уделить качеству архитектуры, обработке ошибок и соблюдению бизнес-правил.

Описание

Необходимо разработать упрощённую систему управления логистическими доставками для компании, которая обеспечивает доставку грузов клиентам.

Система должна позволять:

- регистрировать грузы;
- создавать доставки;
- отслеживать статус доставки;
- назначать транспорт;
- рассчитывать стоимость доставки;
- получать базовую отчётность;
- Уведомлять пользователей о состоянии доставки.

Цель задания

Реализовать backend-приложение на Java для управления доставками в логистической компании.

Технологические ожидания

Рекомендуемый стек:

- Java 17+
- Spring Boot
- Spring Web
- Spring Security
- Spring Data JPA
- PostgreSQL или H2

- Kafka
- REST (Feign)
- Maven или Gradle
- Lombok — по желанию
- Swagger / OpenAPI — желательно
- JUnit / Mockito — желательно

Бизнес-контекст

Компания доставляет грузы из складов в пункты назначения.

Каждая доставка проходит несколько статусов, например:

- CREATED
- STORED
- IN_TRANSIT
- DELIVERED
- CANCELLED

У каждой доставки есть:

- отправитель;
- получатель;
- адрес отправления;
- адрес назначения;
- расстояние;
- вес груза;
- тип транспорта;
- стоимость;
- текущий статус посылки.

Функциональные требования

1. Управление грузами

Нужно реализовать возможность:

- создать груз;
- получить список грузов;
- получить груз по id;
- удалить груз.

2. Управление транспортом

Нужно реализовать:

- создание транспортного средства;
- просмотр списка транспорта;
- просмотр транспорта по id;
- изменение статуса доступности транспорта.

3. Управление доставками

Бизнес-правила:

- нельзя назначить транспорт, если он не **AVAILABLE**;
- нельзя создать доставку, если вес груза больше грузоподъёмности транспорта;
- после назначения транспорта его статус меняется на **IN_USE**;
- после завершения доставки транспорт снова становится **AVAILABLE**;
- отменённая доставка не может быть переведена в другой статус.

Нужно реализовать:

- создание доставки;
- просмотр списка доставок;
- просмотр доставки по id;
- обновление статуса доставки;
- отмену доставки.

4. Расчёт стоимости

Реализовать простой расчёт стоимости доставки.

Пример формулы:

$$\text{стоимость} = \text{базовая_ставка} + (\text{вес} * \text{коэффициент_веса}) + (\text{расстояние} * \text{коэффициент_расстояния})$$

Можно использовать упрощённую модель:

- базовая ставка = 100
- коэффициент веса = 10
- коэффициент расстояния = 5

Расстояние можно передавать как отдельное поле или считать условно заданным числом.

5. Отчётность

Реализовать endpoint'ы для получения:

- количества доставок по статусам;
- списка завершённых доставок;
- общей суммы доставок за период.

6. Нотификация

Реализовать уведомление пользователей о статусе доставки посредством сообщения на почту, номер телефона или любым другим способом. Примеры сообщений: “Заказ 132 создан”, “Доставка заказа 132 начата”, “Заказ 132 доставлен и получен”.

Нефункциональные требования

- соблюдение принципов ООП;
- разделение на слои: controller / service / repository;
- обработка ошибок;
- валидация входных данных;
- понятные названия классов и методов;
- аккуратная структура проекта.

Минимальный результат

- исходный код проекта;
- файл **README.md** с инструкцией по запуску;
- коллекцию Postman или Swagger;
- SQL-скрипт инициализации данных — по желанию;
- тесты — желательно.